

Computer Vision HW1

Toy Race Car Tracker

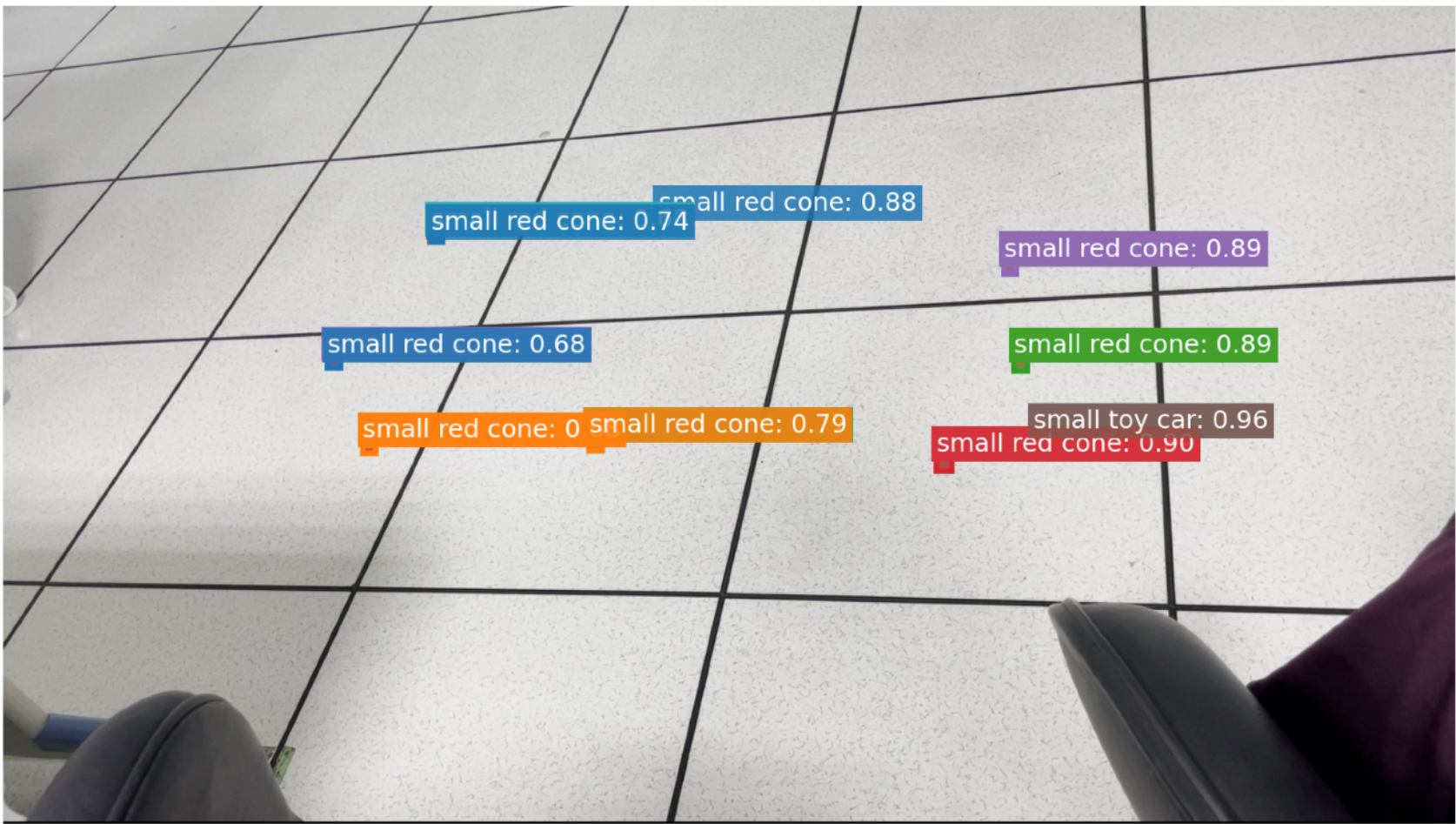
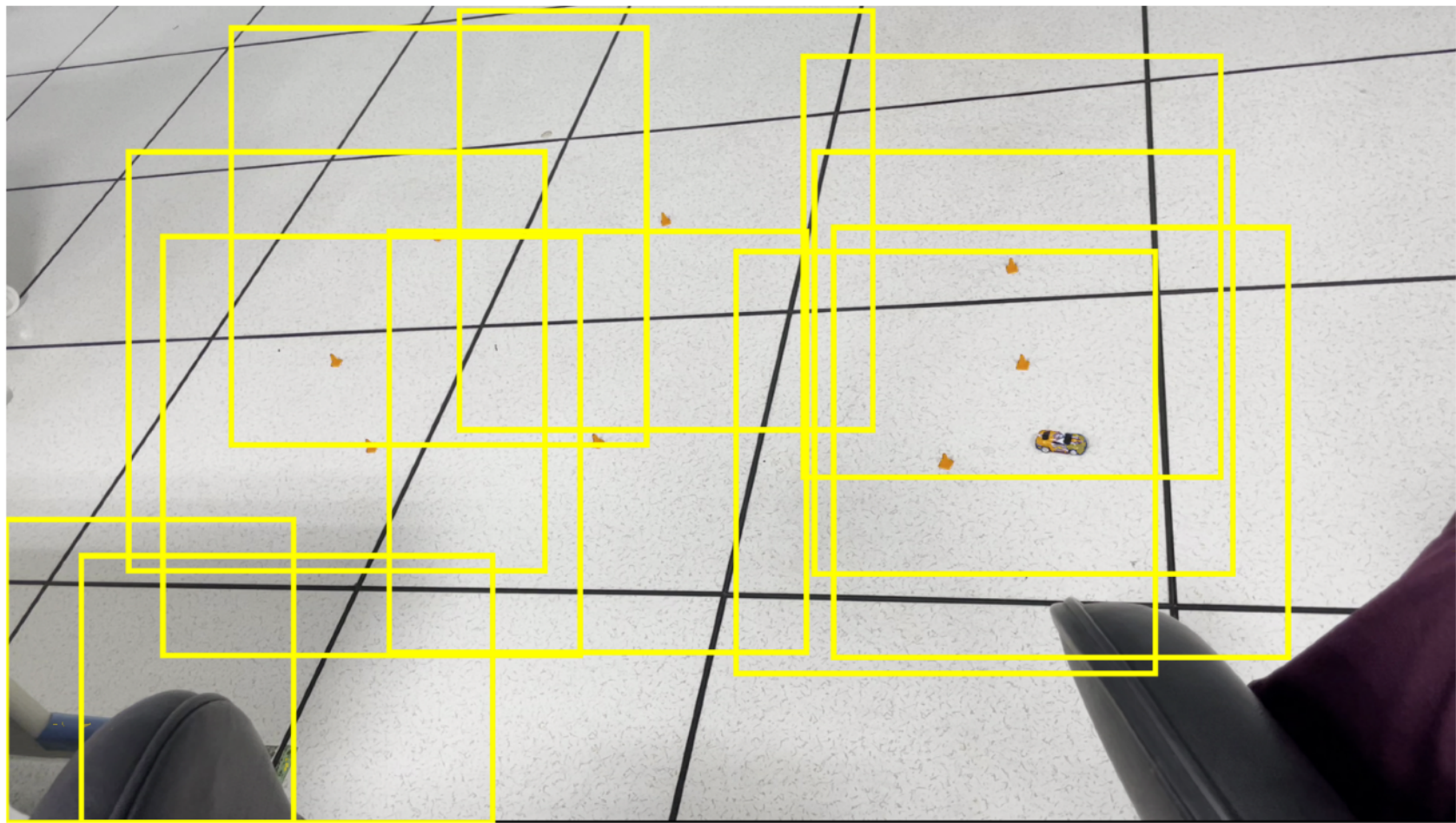
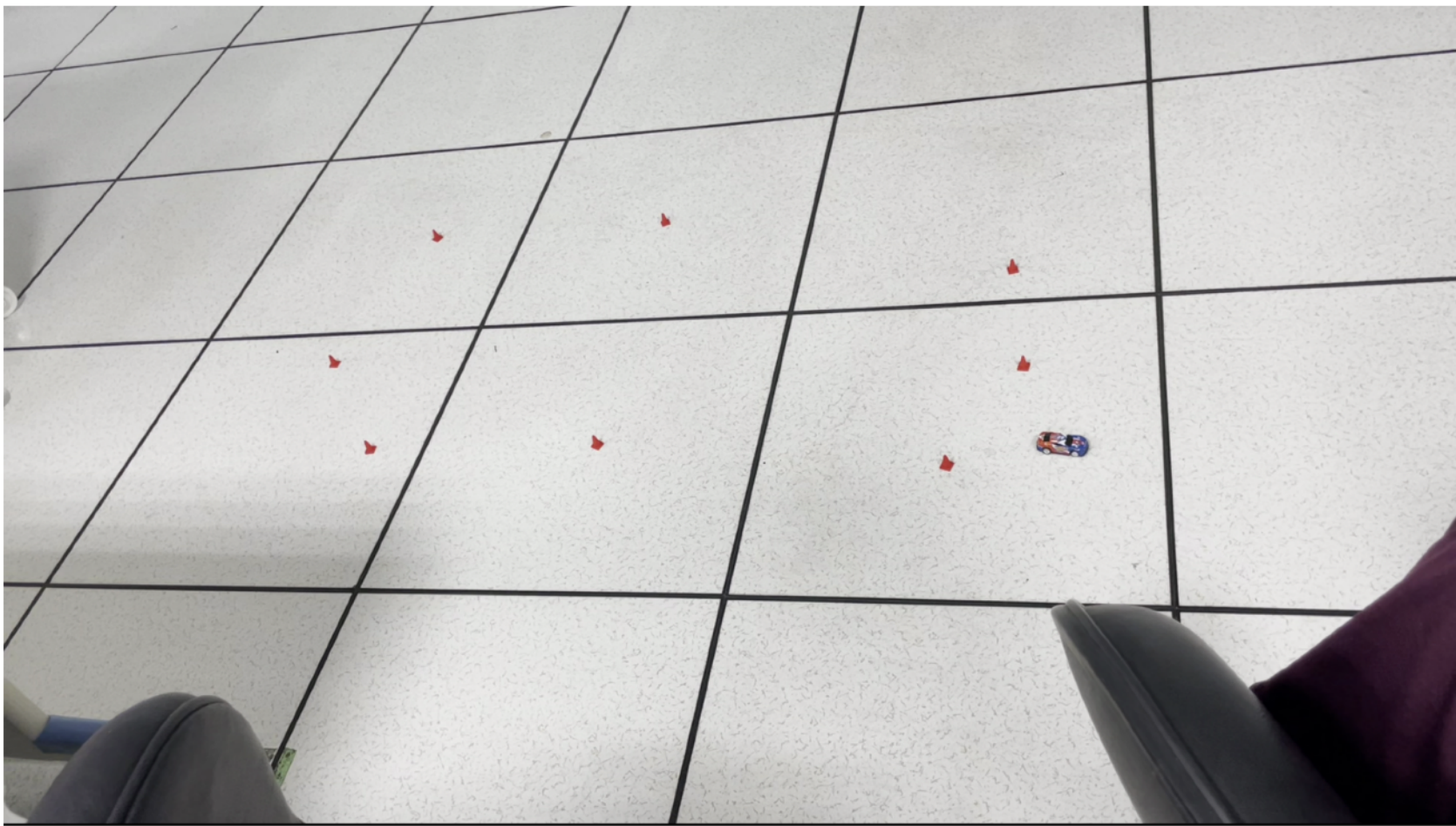
Chieh-Yu Tsai, Yanchen Lin, Cahethel Anais Dounoupetipa, Navdeep Navdeep, Pratham Rakesh Rathod

Data Collection & Detection Pipeline

Original

Filter-selected regions (11)

SAM3: small red cone, small toy car | 26 detection(s) (threshold ≥ 0.50)



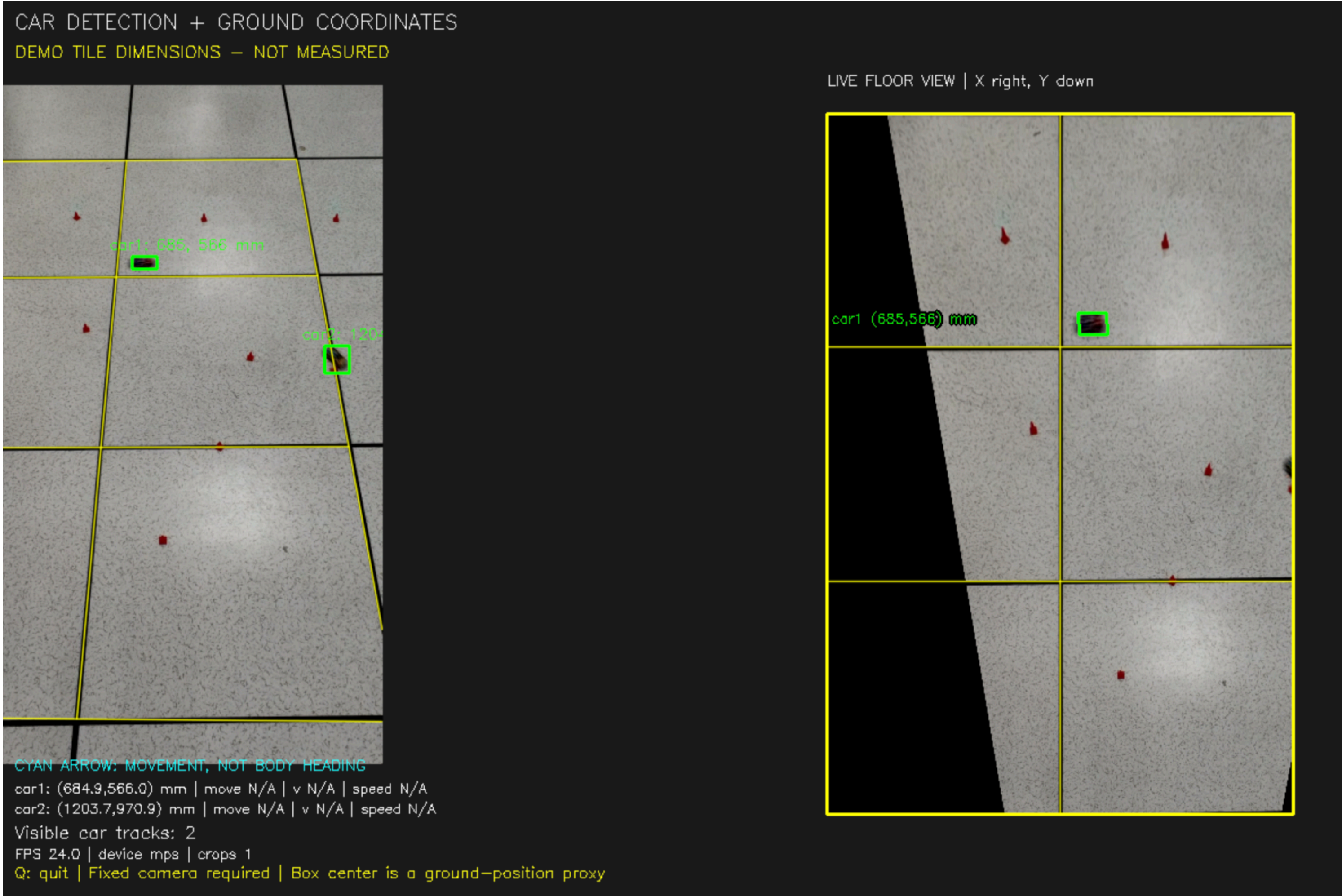
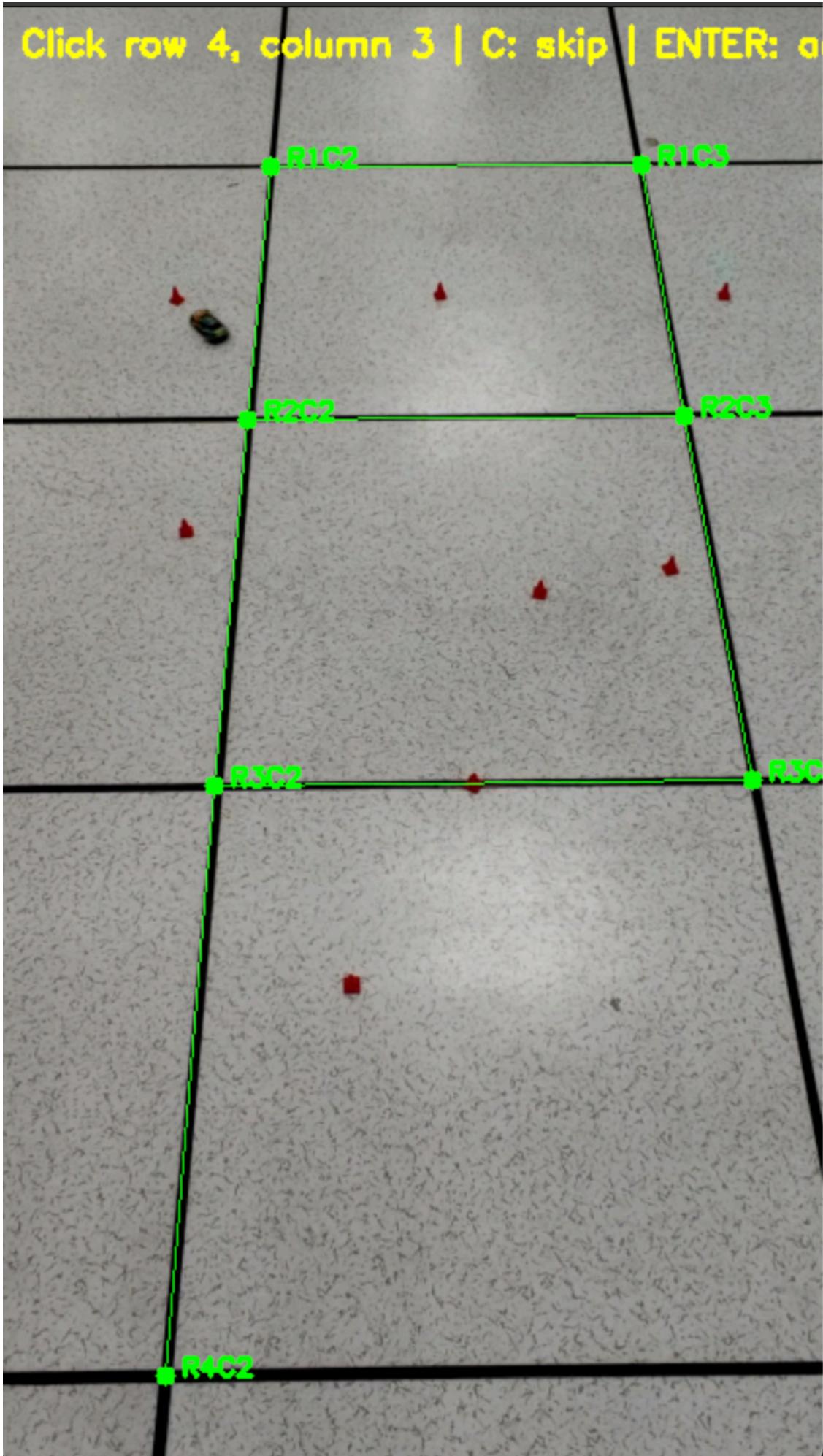
Data collected by SAM

- ID 0 $\rightarrow$ red cone
- ID 1 $\rightarrow$ car

Customized Label Training



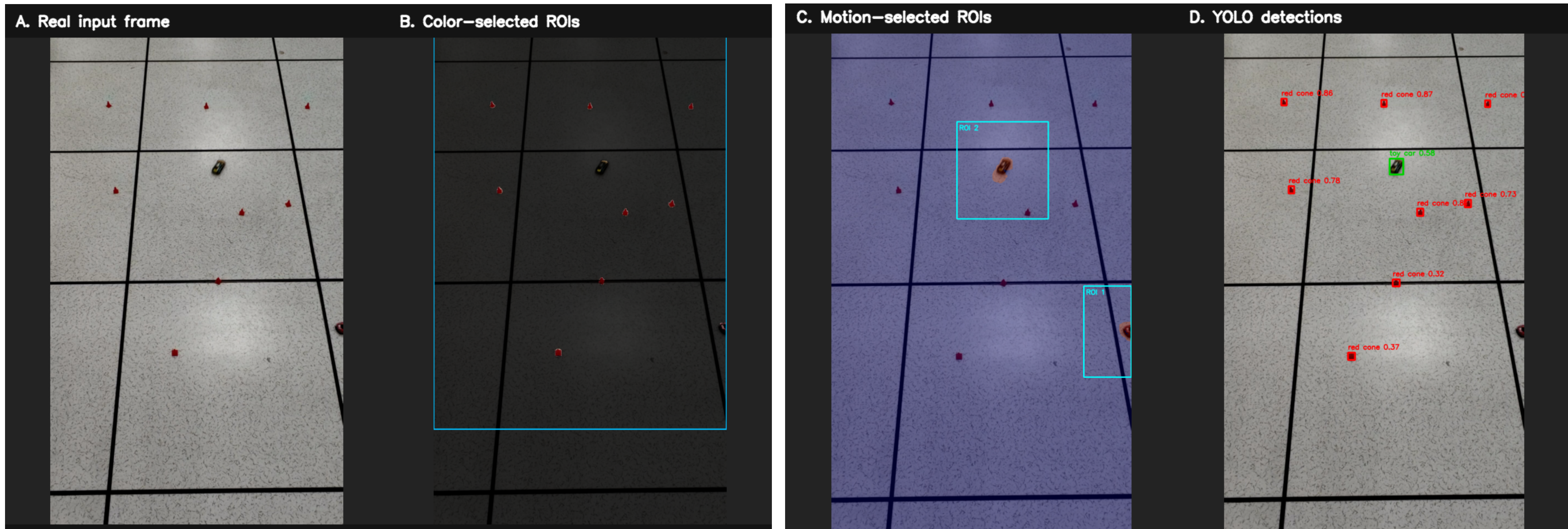
Camera Calibration by Floor Grid



One square represents 600x600 mm, have 2x3 grid. Use this information to do calibration and reproject floor view

Low Latency design for YOLO inference

ROI Extraction → YOLO detection → Temporal Tracking → Pixel-to-World Mapping



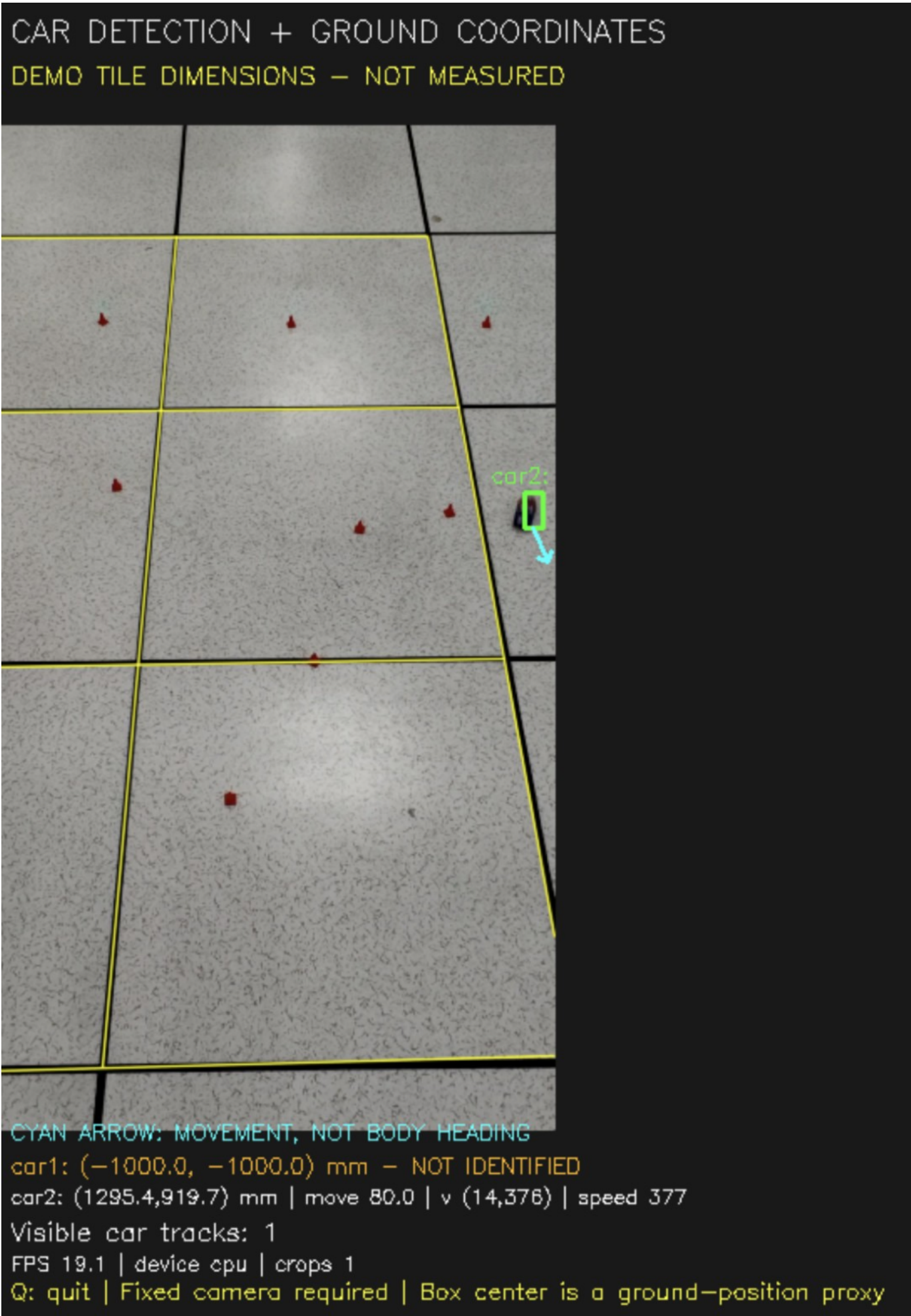
Red cone region selected by color threshold

Car region use motion (frame by frame) to select where is ROIs in large resolution image

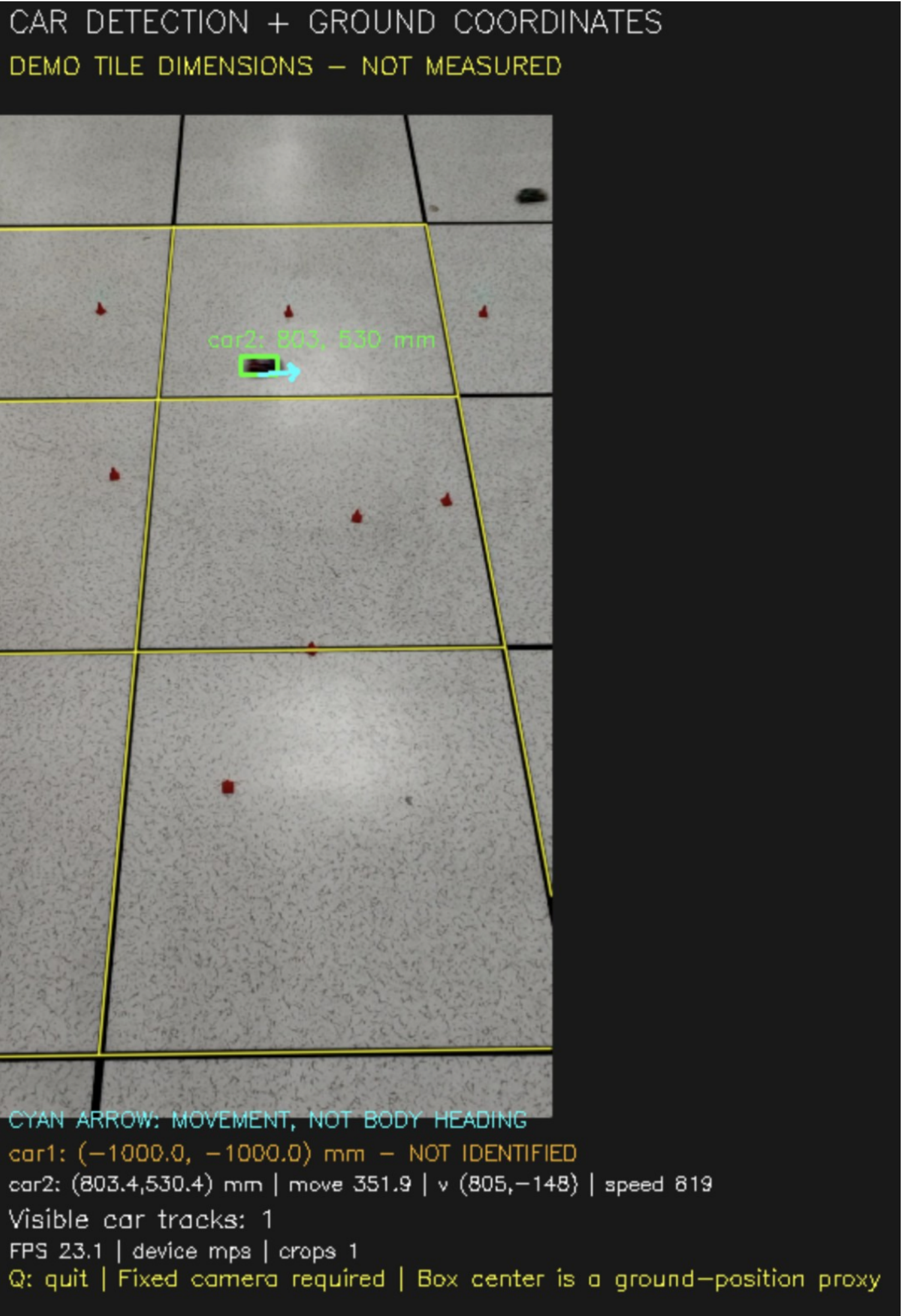
Code Structure

```
.
├── src/car_detection/
│   ├── __main__.py      # `python -m car_detection` entry point
│   ├── cli.py           # Main CLI wrapper
│   ├── pipeline.py      # Calibration, detection, tracking, output
│   ├── config.py        # Typed settings and validation
│   ├── calibration.py   # Snapshot and floor-grid selection
│   ├── rendering.py     # Camera and top-down visualizations
│   ├── geometry/floor.py # Pixel-to-floor transformation
│   ├── tracking/        # Motion and vehicle identity logic
│   └── detector/        # ROI, YOLO, and temporal tracking
├── tests/               # Deterministic unit tests
└── scripts/download_assets.sh # Model/video downloader
```

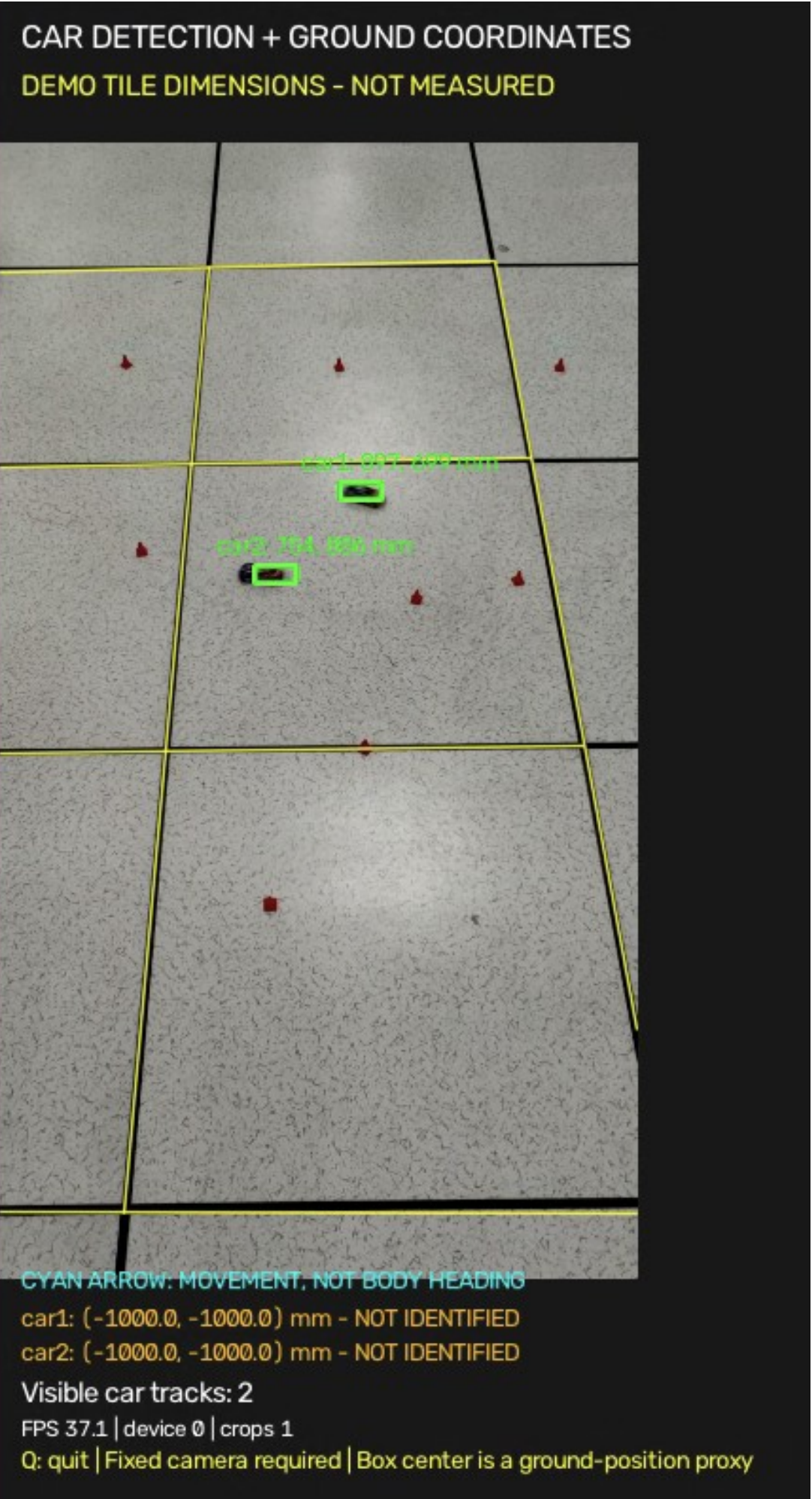
FPS in Different Devices



CPU: 19



MPS: 23



GPU > 30